University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Concession Test, 2025

This test has 5 pages. Make sure you have all of them.
Answer all questions.

Marks per question are shown in brackets.

Time: 45 minutes (+5 minutes reading time) **Marks: 35**

Section 1: Advanced Software Design [17]

Question 1. **[8]**

a) What is the difference between fan-in and fan-out [1]

b) Why is the problem of complexity so much worse in computing than in other engineering disciplines? [1]

c) Consider the following code snippet for the factory pattern
Why should you **not** use the `default` clause of the case/switch statement (line 7)?

```
1 public Shape shapeFactory(ShapeEnum shapeEnum, String params)
2 throws UnknownShapeException {
3   switch (shapeEnum) {
4    case ID_oval: return new RoundRect(params, 100);
5    case ID_rect: return new RoundRect(params, 0);
6    case ID_blob: return new Blob(params);
7    //default: don't do it!
8  } //switch
9  throw new UnknownShapeException(shapeEnum, params); }
```
[1]

d) In recent years, the focus in dev-ops has shifted from shift-right to shift left.
What does shift-left mean? [2]

e) In a previous life, or perhaps in a recurring nightmare, I was a sysadmin. In those days websites used unsecured http and you could use a proxy like Squid to cache traffic or filter out pages. Nowadays, websites use https and proxies are no longer possible.
Name the security problem that this change protects against
and explain how this security threat works
(you may draw a diagram, but must also explain in prose how it works) [3]

Question 2. **[4]**

Consider the following pseudocode snippet

```
1  const int clNone = 0;
2  const int clRed = 31;
3  const int clBlue = 34;
4
5  interface IShape:
6    public:
7      virtual string get_str()
8
9  class Circle : public IShape:
10   public:
11     float radius
12     Circle(float r): radius = r
13     string get_str() override:
14       return format("A Circle of radius {}", radius)
15
16 class Square : public IShape:
17   public:
18     float side
19     Square(float s): side = s
20     string get_str() override:
21       return format("A Square with side {}", side)
22
23 class ColorShape : public IShape:
24   private:
25     //https://iq.opengenus.org/print-text-in-color-in-c/
26     const string colorCode = "\033[1;{}m" //ansi colors
27     int color
28     IShape& shape
29  public:
30     ColorShape(const int c, IShape& s):color = c; shape = s
31     string get_str() override:
32       const string cl_on = format(colorCode, color)
33       const string cl_off = format(colorCode, ColorNone)
34       return format("{}{} now colored {}"
35         ,cl_on, shape.get_str(), cl_off);
36
37 int main():
38   Square square(5)    // A square with side length 5
39   Circle circle(10)   // A circle with radius 10
40   //add color
41   ColorShape redSquare(ColorRed, square)
42   ColorShape blueCircle(ColorBlue, circle)
43   println("{}", redSquare.get_str())
44   println("{}", blueCircle.gte_str())
45   println("{}", square.get_str())
```

a) What is the name of this design pattern? [1]

b) What does this pattern do, how does it? [2]

c) Pick one (1) of the SOLID principles that applies to this pattern and explain how using this pattern makes your code SOLID [1]

Question 3. **[5]**

a) In AI, both reasoning (R) and perception (P) can be used to generate answers given a question phrased in a format that either technique can understand.
Stated like that both systems seem much alike.
In terms of the answers generated, what is the difference between R and P and why does this difference manifest itself? [2]

b) The Scrum method heavily depends on the positive assumptions of human work motivation and management known as 'theory Y'.
Explain briefly how applying this principle in Scrum makes team members produce high quality code. [2]

c) I have a parrot. It talks... a lot.
Why are 'chickens' not allowed to talk in Scrum meetings? [1]

Section 2: Theory of Algorithms [9]

Question 4. **[2]**

You are told that the time complexity of a particular algorithm is

$$T(n) = 5\ T(n/5) + n^2$$

for n inputs.

Derive the complexity class of this algorithm.

Question 5. **[2]**

Use Horner's algorithm for exponentiation to find the *number of multiplies* needed to compute a^7. Show your working.

Question 6. **[2]**

Generate the Horspool Table for the pattern **RATTLE** assuming A-Z as the underlying alphabet.

(Please see next page for question 7)

Question 7. **[3]**

You are given the following solution table for the knapsack problem of capacity W = 3 and n=4 items, with the indicated weights (w_i) and values (v_i) for the ith item:

	Capacity (j)				
		0	**1**	**2**	**3**
Items (n)	**0**	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12
w_2=1, v_2=10	**2**	0	10	12	22
w_3=3, v_3=20	**3**	0	10	12	22
w_4=2, v_4=15	**4**	0	10	15	**25**

Use this information determine the *most valuable items* that can fit into the knapsack of capacity 3. Show your reasoning – based on table values - used to determine the solution set – guess will not be accepted.

Section 3: Theory of Computation [9]

Question 8. **[9]**

a) Using B to represent a blank cell, draw a transition diagram for a Turing Machine with alphabet {B,1} that ***accepts even unary numbers***. A unary number N is represented by a string of N+1 consecutive 1s. The tape head will start on a blank cell, followed immediately by the number. The machine must stop on the first blank cell after the unary number if it is even, and must crash if it is not even. Example input for unary 2 (tape head shown below the tape):

B111BB- - -
↑

which, because 2 is even, must halt with the tape head at the first blank after its input:

B111BB- - -
↑ [4]

b) Give an example of any one row in your Turing Machine's State Table. [1]

c) Explain what is meant by the *lower bound* and the *upper bound* of a problem. [2]

d) We showed that the Element Uniqueness problem (call it U) is reducible to the Minimum Spanning Tree problem (call it T) in polynomial time. Which of the following statements follow from this? Give the statement number, and a brief reason for your answer.

1. The lower bound of U is no worse than the lower bound of T (U is no harder than T)
2. The lower bound of T is no worse than the lower bound of U (T is no harder than U)

[2]

Appendix A – Master Theorem

If we have a recurrence of this form: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$